Timing attacks

A timing attack works by measuring minute differences in the time it takes a computer to process different inputs to work out some information about a secret value that the attacker does not know. Timing attacks can measure even very small differences in the time it takes to perform a computation, even when carried out over the internet. The classic paper *Remote Timing Attacks are Practical* by David Brumley and Dan Boneh of Stanford (2005, https://crypto.stanford.edu/~dabo/papers/ssl-timing.pdf) demonstrated that timing attacks are practical for attacking computers on the same local network, and the techniques have been developed since then.

Consider what would happen if you used the normal `String equals` method to compare the hash of the session ID with the anti-CSRF token received in a header. In most programming languages, including Java, string equality is implemented with a loop that terminates as soon as the first non-matching character is found. This means that the code takes very slightly longer to match if the first two characters match than if only a single character matches. A sophisticated attacker can measure even this tiny difference in timing. They can then simply keep sending guesses for the anti-CSRF token. First, they try every possible value for the first character (64 possibilities because we are using base64-encoding) and pick the value that took slightly longer to respond. Then they do the same for the second character, and then the third, and so on. By finding the character that takes slightly longer to respond at each step, they can slowly recover the entire anti-CSRF token using time only proportional to its length, rather than needing to try every possible value. For a 10-character Base64-encoded string, this changes the number of guesses needed from around 64^{10} (over 1 quintillion possibilities) to just 640. Of course, this attack needs many more requests to be able to accurately measure such small timing differences (typically many thousands of requests per character), but the attacks are improving all the time.

The solution to such timing attacks is to ensure that all code that performs comparisons or lookups using secret values take a constant amount of time regardless of the value of the user input that is supplied. To compare two strings for equality, you can use a loop that does not terminate early when it finds a wrong value. The following code uses bitwise XOR (^) and OR (|) operators to check if two strings are equal. The value of c will only be zero at the end if every single character was identical.

```
if (a.length != b.length) return false;
int c = 0;
for (int i = 0; i < a.length; i++)
   c |= (a[i] ^ b[i]);
return c == 0;
```

This code is very similar to how `MessageDigest.isEqual` is implemented in Java. Check the documentation for your programming language to see if it offers a similar facility.